

Livrable 5 — Industrialisation d'un algorithme d'apprentissage automatique et automatisation des processus de décision

Livrable attendu : Code source contenant la création de l'environnement standardisé, le déploiement de l'algorithme, l'application web ainsi que l'URL de l'application déployée.

1. Table des matières

- [1. Table des matières](#)
- [2. Objectif du livrable](#)
- [3. Vue d'ensemble de l'architecture de déploiement](#)
- [4. Standardisation de l'environnement](#)
- [5. Industrialisation du modèle d'apprentissage](#)
- [6. Exposition du modèle par une API REST](#)
- [7. Application web de démonstration](#)
- [8. Industrialisation du déploiement](#)
- [9. Déploiement local et reproductibilité](#)
- [10. Ressources du projet](#)
- [11. Conclusion](#)

2. Objectif du livrable

Ce livrable présente l'industrialisation de la couche applicative développée dans le cadre du projet de transcription automatique de guitare.

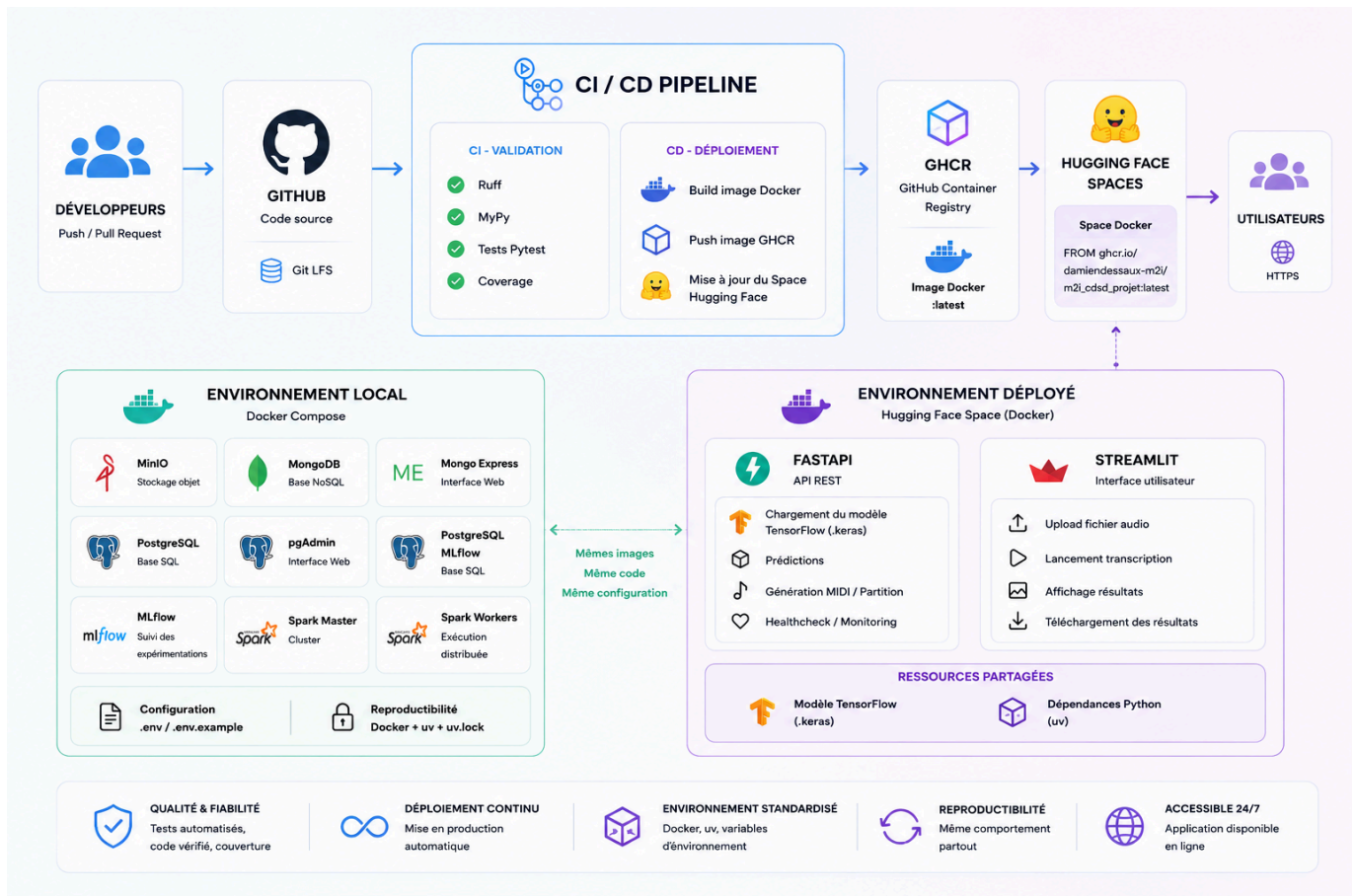
Après avoir été entraîné et sélectionné lors des expérimentations de Machine Learning, le modèle est intégré dans une architecture logicielle permettant son exploitation en production. Cette architecture poursuit trois objectifs :

- standardiser l'environnement d'exécution afin de garantir la reproductibilité des déploiements,
- exposer les capacités du modèle au travers d'une API REST documentée,
- mettre à disposition une application web accessible aux utilisateurs finaux.

L'ensemble de la chaîne de déploiement est entièrement automatisé grâce à une pipeline CI/CD, depuis la validation du code jusqu'à la mise en production sur Hugging Face Spaces.

3. Vue d'ensemble de l'architecture de déploiement

La figure suivante présente l'architecture de déploiement retenue pour la couche applicative.



L'industrialisation débute après la phase d'expérimentation des modèles. Une fois le meilleur modèle **TensorFlow** sélectionné et exporté, celui-ci est intégré à l'API FastAPI.

À chaque évolution du dépôt GitHub, une chaîne d'intégration continue valide automatiquement la qualité du projet avant de construire une image Docker unique. Cette image est publiée sur **GitHub Container Registry (GHCR)** puis utilisée pour mettre à jour automatiquement le **Hugging Face Space** hébergeant l'application.

L'utilisateur interagit uniquement avec l'interface Streamlit. Celle-ci communique avec l'API REST afin de déclencher la transcription audio et de restituer les différents artefacts générés (MIDI, partition, piano roll).

Cette architecture garantit un environnement identique entre le développement, l'intégration continue et la production.

4. Standardisation de l'environnement

L'un des objectifs de ce projet consiste à garantir que l'application puisse être exécutée de manière identique quel que soit son environnement d'hébergement.

Pour répondre à cette exigence, la totalité de la couche applicative est distribuée sous la forme d'une image Docker unique. Cette image est utilisée :

- lors du développement local,
- dans la chaîne CI/CD GitHub Actions,
- sur la plateforme de déploiement Hugging Face Spaces.

L'environnement Python est géré avec **uv**, qui assure une installation rapide et déterministe des dépendances. La configuration du projet est décrite dans le fichier **pyproject.toml**, tandis que le fichier **uv.lock** verrouille précisément les versions installées afin de garantir la reproductibilité des environnements.

Le conteneur embarque notamment :

- Python 3.13,
- **FastAPI**,
- **Pydantic**,
- **Streamlit**,
- **TensorFlow**,
- **music21**,
- **Verovio**,
- **CairoSVG**,
- l'ensemble des dépendances système nécessaires au traitement audio et à la génération des partitions.

La construction de l'image repose sur un **Dockerfile multi-stage**, permettant de limiter la taille finale de l'image tout en accélérant les temps de construction. Un utilisateur non privilégié est créé afin de renforcer la sécurité du conteneur conformément aux bonnes pratiques Docker.

Enfin, le script **start.sh** assure le démarrage simultané de l'API **FastAPI** et de l'interface **Streamlit** dans un même conteneur. Ce choix simplifie le déploiement de l'application tout en conservant une séparation logique entre la couche de présentation et la couche de services.

L'utilisation conjointe de **Docker**, **uv** et d'une gestion stricte des dépendances garantit ainsi un environnement standardisé, reproductible et portable.

5. Industrialisation du modèle d'apprentissage

Le modèle de Deep Learning utilisé par l'application est développé indépendamment de la couche applicative. Son cycle de vie est volontairement dissocié de celui de l'API afin de faciliter les expérimentations et les futures évolutions du modèle.

Les différentes architectures de réseaux de neurones sont entraînées puis comparées au cours d'une phase d'expérimentation. Chaque entraînement est suivi avec **MLflow**, qui conserve les paramètres d'apprentissage, les métriques d'évaluation ainsi que les artefacts produits.

À l'issue de cette phase, le modèle retenu est exporté au format **TensorFlow (.keras)** puis intégré manuellement au projet applicatif.

L'API n'effectue aucun entraînement. Son rôle se limite exclusivement à l'exécution de la chaîne d'inférence :

1. chargement du modèle lors du démarrage de l'application,
2. prétraitement du fichier audio,
3. extraction des caractéristiques musicales,
4. inférence du réseau de neurones,
5. post-traitement des prédictions,
6. génération des fichiers MIDI, des partitions musicales et des représentations graphiques.

Afin d'optimiser les performances, le modèle est chargé une seule fois au démarrage de FastAPI grâce au mécanisme de **lifespan**. L'instance ainsi créée est ensuite partagée entre toutes les requêtes grâce au système d'injection de dépendances, évitant tout rechargement inutile du modèle.

Cette organisation permet de réduire les temps de réponse tout en limitant la consommation mémoire de l'application.

6. Exposition du modèle par une API REST

Afin de rendre le modèle de Deep Learning exploitable par d'autres applications et par des utilisateurs métiers, celui-ci est exposé au travers d'une **API REST** développée avec **FastAPI**.

Cette API constitue le point d'entrée unique de l'ensemble des traitements. Elle encapsule toute la logique métier nécessaire à la transcription automatique d'un fichier audio et permet de dissocier complètement le moteur de prédiction de l'interface utilisateur.

L'architecture logicielle repose sur une séparation claire des responsabilités. Le projet distingue notamment :

- les **routes HTTP**, responsables de l'exposition des services REST,
- les **services métier**, qui implémentent les traitements de transcription,
- les **modèles Pydantic**, utilisés pour la validation et la sérialisation des données,
- les **dépendances FastAPI**, qui assurent l'injection des composants techniques,
- les **composants d'infrastructure**, responsables notamment du chargement du modèle et de la configuration de l'application.

Cette organisation améliore la lisibilité du code, facilite les tests unitaires et limite le couplage entre les différentes couches de l'application.

Le modèle **TensorFlow** est chargé une seule fois lors du démarrage de l'application grâce au mécanisme **lifespan** de FastAPI. Il est ensuite partagé entre toutes les requêtes au moyen de l'injection de dépendances, garantissant ainsi des temps de réponse constants et une utilisation optimisée de la mémoire.

L'API expose plusieurs services REST permettant de couvrir l'ensemble du processus de transcription :

Route	Fonction
/health	Vérification de l'état de l'application et du modèle
/model	Consultation des informations du modèle chargé
/predict	Lancement d'une transcription audio
/artifact/{id}/midi	Téléchargement du fichier MIDI
/artifact/{id}/piano_roll/png	Téléchargement du piano roll (PNG)
/artifact/{id}/piano_roll/svg	Téléchargement du piano roll (SVG)
/artifact/{id}/score/pdf	Téléchargement de la partition (PDF)
/artifact/{id}/score/svg	Téléchargement de la partition (SVG)

Toutes les réponses sont normalisées au travers d'un modèle générique `ApiResponse<T>`, garantissant une structure homogène pour les réponses de succès comme pour les erreurs.

Les exceptions sont centralisées grâce à des gestionnaires dédiés, permettant de produire des messages d'erreur cohérents tout en simplifiant la maintenance de l'application.

L'utilisation de FastAPI permet également de générer automatiquement une documentation OpenAPI interactive, facilitant l'intégration de l'API par d'autres applications.

Cette API répond ainsi à la compétence **C5.2** du référentiel en mettant à disposition un service de prédiction standardisé, documenté et réutilisable.

7. Application web de démonstration

Bien qu'une API REST puisse être utilisée directement par des applications tierces, elle reste peu adaptée à des utilisateurs non techniques. Une interface web a donc été développée avec **Streamlit** afin de rendre le modèle immédiatement exploitable.

Cette interface communique exclusivement avec l'API REST, sans accéder directement au modèle de Deep Learning. Ce choix garantit une séparation claire entre la couche de présentation et la logique métier.

Le parcours utilisateur se déroule selon les étapes suivantes :

1. dépôt d'un fichier audio au format **WAV**,
2. envoi de la requête à l'API REST,
3. exécution de la chaîne complète d'inférence,
4. affichage des résultats de la transcription,
5. téléchargement des artefacts générés.

L'application permet notamment de récupérer :

- le fichier MIDI généré,
- la représentation graphique du piano roll (PNG ou SVG),
- la partition musicale (PDF ou SVG).

L'ensemble des traitements est exécuté côté serveur. L'interface Streamlit se limite à la collecte des données utilisateur, à l'appel des services REST et à la restitution des résultats.

Cette architecture facilite les évolutions futures, puisque toute nouvelle interface (application mobile, client desktop ou autre interface web) pourra réutiliser la même API sans modification du moteur de prédiction.

8. Industrialisation du déploiement

Le projet s'appuie sur une chaîne **CI/CD** entièrement automatisée reposant sur **GitHub Actions**.

Chaque **Push** ou **Pull Request** déclenche automatiquement une succession d'étapes garantissant la qualité du code avant son déploiement.

La phase d'intégration continue comprend :

- l'installation reproductible de l'environnement Python avec **uv**,
- l'exécution des tests unitaires et d'intégration avec **Pytest**,
- la mesure du taux de couverture de tests avec validation d'un seuil minimal,
- l'analyse statique du code avec **Ruff**,
- la vérification du typage grâce à **MyPy**.

Les rapports de couverture sont publiés sous forme d'artefacts **GitHub Actions** afin de conserver un historique des validations réalisées.

Lorsque l'ensemble de ces contrôles est validé, la phase de livraison continue est automatiquement exécutée.

Elle consiste à :

1. construire l'image **Docker** de l'application,
2. publier cette image dans **GitHub Container Registry (GHCR)**,
3. mettre à jour automatiquement le Dockerfile du Space **Hugging Face** afin qu'il référence la dernière image publiée,
4. déployer automatiquement la nouvelle version de l'application.

Cette automatisation réduit les interventions manuelles, garantit la reproductibilité des déploiements et assure que seule une version validée du projet est mise en production.

L'intégration continue et la livraison continue constituent ainsi un élément essentiel de l'industrialisation de la solution.

9. Déploiement local et reproductibilité

L'application peut être déployée localement afin de reproduire un environnement identique à celui utilisé en intégration continue et en production.

Après avoir créé le fichier de configuration à partir du modèle fourni, l'ensemble des services est démarré à l'aide de Docker Compose.

```
# Création du fichier .env
cp .env.example .env

# Construction de l'infrastructure
docker compose up -d
```

Le fichier **docker-compose.yml** orchestre les différents services nécessaires au fonctionnement du projet, notamment :

- l'API **FastAPI**,
- l'interface utilisateur **Streamlit**.

Cette approche garantit que tous les développeurs disposent d'un environnement identique, limitant les écarts entre les phases de développement, de validation et de production.

Une fois les conteneurs démarrés, les principaux points d'accès sont les suivants :

Service	Adresse
Interface utilisateur	http://localhost:7860
API REST	http://localhost:8000
Documentation OpenAPI	http://localhost:8000/docs

L'utilisation de Docker Compose complète ainsi la démarche de standardisation en permettant de reconstruire l'ensemble de l'environnement applicatif à partir du seul code source.

10. Ressources du projet

L'application déployée est accessible publiquement :

Hugging Face Spaces : https://huggingface.co/spaces/DamienDESSAUX/M2i_CDSD_Projet_Deployment

Elle exactement à l'image Docker construite et publiée automatiquement par la chaîne CI/CD.

11. Conclusion

Ce projet met en œuvre une chaîne complète d'industrialisation d'un algorithme d'apprentissage automatique, depuis son intégration dans une application jusqu'à son déploiement automatisé en production.

L'environnement d'exécution est entièrement standardisé grâce à **Docker** et **uv**, garantissant des installations reproductibles entre les postes de développement, la chaîne CI/CD et la plateforme de production. Le modèle de Deep Learning est intégré dans une **API REST FastAPI**, documentée automatiquement via OpenAPI et organisée selon une architecture en couches favorisant la maintenabilité, les tests et l'évolutivité. Une interface **Streamlit** permet aux utilisateurs de soumettre un fichier audio, de lancer une transcription et de récupérer les artefacts générés sans connaissance technique particulière.

L'industrialisation est complétée par une chaîne **GitHub Actions** assurant automatiquement les contrôles qualité (tests, couverture, analyse statique et vérification du typage), la construction de l'image Docker, sa publication sur **GitHub Container Registry (GHCR)** et son déploiement sur **Hugging Face Spaces**. Cette automatisation garantit que chaque version mise à disposition des utilisateurs provient d'un code validé et exécuté dans un environnement maîtrisé.

L'ensemble de cette architecture constitue une solution d'industrialisation cohérente, reproductible et conforme aux bonnes pratiques actuelles du développement logiciel et du Machine Learning en production.

Auteur : Damien DESSAUX